THESIS DEFENSE

PRISM : Queue Aware Pull Request Prioritization and Reviewer Recommendation

Niaz Rahman - 210042159

Antara Arifa Mullick - 210042132

Labibah Hoque - 210042121

Network & Data Analysis Group

Supervisor : Lutfun Nahar Lota, Assistant Professor

Co-Supervisor : Dr. Md. Azam Hossain, Associate Professor

Deptartment of Computer Science & Engineering
Islamic University of Technology

BACKGROUND

Pull Review Request

Pull request are the primary mechanism for proposing and Integrating code changes in modern software Projects

The Review Bottleneck

However, as repositories grow, more pull requests compete for a limited pool of experienced reviewers.This creates review queues where turnaround depends not only on reviewer suitability, but also on who is available and how quickly they can respond.



This leads us to two important decisions:
First, which pull request should be reviewed first? This is the pull request prioritization problem.
Second, who should review each pull request? This is the reviewer recommendation problem.

LITERATURE REVIEW

RevFinder

A File Location Based Reviewer Recommendation Approach

Thongtanunam et al. (2015)

1. New Pull Request

Extract the set of changed file paths from the PR.

```
src/auth/login.py
src/utils/validate.py
...
```

→

2. Historical Reviews

Retrieve past reviews of similar files and directories from the repository history.

```
Developer A
Developer B
Developer C
...
```

→

3. File Path Similarity

Compute similarity between the changed file paths and historical file paths using four string-based techniques.

Prefix	Common prefix match
Suffix	Common suffix match
Substring	Substring match
Subsequence	Subsequence match

→

4. Rank Reviewers

Aggregate similarity scores across files and rank developers to recommend top reviewers.

Ranked Reviewers

1. Developer A
2. Developer B
3. Developer C

...

Dataset & Evaluation

- 4 open source projects (Android, OpenStack, Qt, LibreOffice)
- Evaluated using historical pull request data

Key Result

69% – 87%

Top-10 accuracy (across projects)

Key Takeaway

File location similarity can effectively identify suitable reviewers, showing the importance of repository structure and past review history.

LITERATURE REVIEW

RCCFinder

Automated Code Reviewer Recommendation for Pull Requests

e-Informatica 2024 | Moosareza & Heydarnoori

1 Approach

Combines multiple repository signals to score and rank suitable reviewers.

Review expertise	Based on past reviews of similar files and directories.
Commit expertise	Based on commit history in relevant parts of the project.
Collaboration signal	Uses co-review and social connections between developers.
Recency handling	Applies cube root time decay so that recent activity has higher weight.

2 Key Results

Evaluated on real projects from multiple open source systems (Apache Flink).

Metric	Performance
Top 1	43%
Top 3	72%
Top 5	82%
Top 10	91%
Mean Reciprocal Rank (MRR)	0.59

Evaluated using historical pull request data from Apache Flink.

3 Insights & Limitations

What it shows

- Combining multiple repository signals can produce effective reviewer recommendations.
- Using recency helps capture current expertise.

Key limitation

- Focuses only on the reviewer selection task.
- Does not consider the dynamic state of the review process (e.g., reviewer workload, ongoing reviews, or queue interactions).
- Treats each pull request independently.

LITERATURE REVIEW

ARRPriotizer

Prioritizing Pull Requests through Dual Prediction of Acceptance and Integrator Response

Khan et al. | Spectrum of Engineering Sciences, 2025 | Published: 25 July 2025

1 Approach

Uses two XGBoost classifiers to predict acceptance likelihood and integrator response time for each pull request.

Acceptance classifier
Predicts probability that a PR will be accepted.

Integrator response classifier
Predicts probability of a response within 24 hours.

→

Combined urgency score
The two probabilities are combined into one urgency score.

2 Key Results

Evaluated on a large set of real-world data from multiple open source projects.

278,418 PRs	41
Total pull requests	GitHub projects

AUC	AUC
0.919	0.893
Acceptance prediction	Integrator response prediction

Outperformed single signal baselines.

3 What the paper contributes

1. **Combines acceptance and responsiveness**
   Uses two predictive signals together to estimate PR urgency.
2. **Ranks PRs by predicted urgency**
   Produces an ordered list of pull requests based on the combined score.
3. **Makes prioritization more actionable**
   Helps integrators focus on PRs that are both likely to be accepted and to receive a timely response.

Key limitation
The system ranks PRs, but reviewer selection remains outside the model.

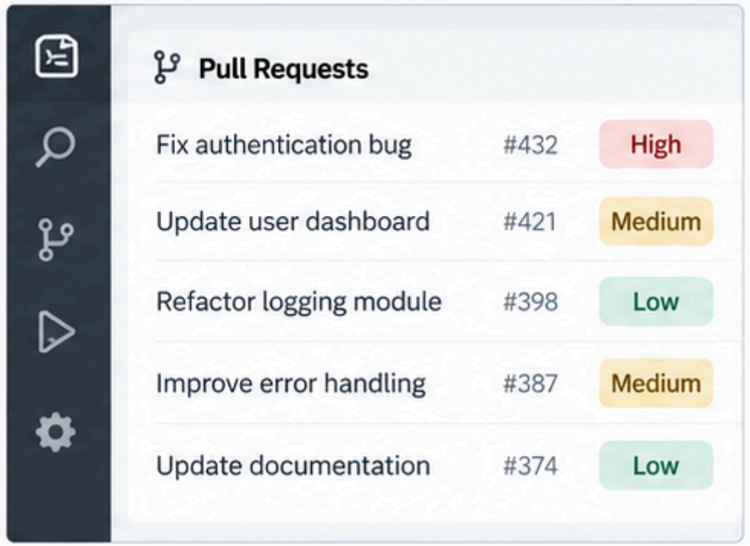
LITERATURE REVIEW

PRECOG: Pull Request Prioritization and Visualization

SANER 2026 | Raskin, Devroey & Vanderose

1 Method

A Visual Studio Code plugin that ranks pull requests using predicted review difficulty.



Example view in the VS Code plugin

2 Key Findings

User study with 9 developers on the Java Spring Framework.

9 Developers

Java Spring Framework Project

Feedback from participants

> *Participants agreed the view helped them make rapid, informed decisions.*

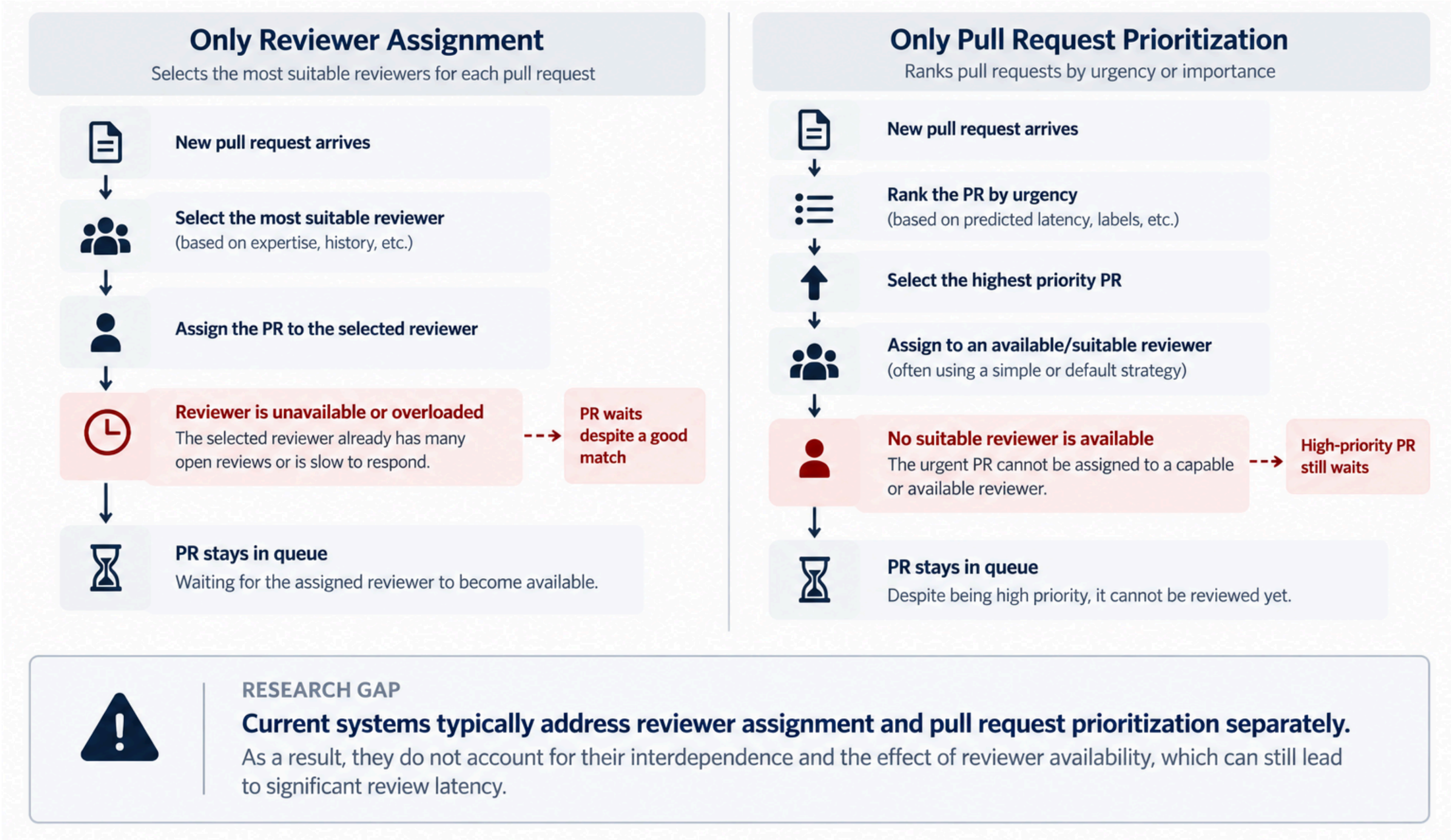
3 Why it Matters Here

1. **Shows the value of prioritization**
   Demonstrates that predicted review difficulty can help reviewers choose what to review first.
2. **Focuses only on PR ordering**
   Does not address reviewer assignment or reviewer workload.
3. **Motivates our work**
   Highlights the need to go beyond PR prioritization and consider both reviewer recommendation and queue awareness.

COMPARISON OF EXISTING APPROACHES

Approach	Reviewer Assignment	PR Ordering	Core idea / method
RevFinder	YES	NO	File location similarity between changed files and previously reviewed files.
RCC Finder	YES	NO	Heuristic Arrproach that combines reviewer expertise, commit experience, collaboration, and recency.
ARRPrioritizer	NO	YES	Uses acceptance and integrator response time to rank PRs by urgency.
PRECOG	NO	YES	Uses review difficulty to decide which PR should be reviewed first.

RESEARCH GAP



PROBLEM STATEMENT

Existing approaches often consider **PR prioritization** and r**eviewer assignment** separately, without accounting for **changing reviewer workload** and **queue conditions**.

RESEARCH CONTRIBUTION

Introducing PRISM

A queue aware framework that jointly prioritizes pull requests and assigns reviewers while updating reviewer workload dynamically.

01 Joint decision pipeline

A seven stage pipeline combines PR ordering and reviewer assignment instead of treating them as separate tasks.

02 Queue aware responsiveness

Reviewer selection accounts for current workload and availability as the queue is processed.

03 Rank based prioritization

PR priority combines predicted latency and the best available reviewer suitability using percentile ranks.

RESEARCH QUESTIONS

RQ1 How does the choice of reviewer affect review turnaround when queue order is fixed?

RQ2 Does more accurate reviewer matching lead to faster pull request reviews?

RQ3 Does jointly considering PR prioritization and reviewer assignment improve review turnaround, and what dependencies arise between the two decisions?

EXPERIMENTAL SETUP: DATASET

10,819
Pull requests

925
Candidate reviewers

1,557
Evaluated pull requests

80 / 20
Temporal split

EXPERIMENTAL SETUP : MODELS

Dataset & Temporal Split

Apache Flink (GitHub) | 2014 – 2020

10,819	925	80%	20%	1,557
PRs	reviewers	training (8,655 PRs)	held out (2,164 PRs)	evaluation PRs (2,439 rows)

Temporal Leakage Control

All historical features are computed using only events strictly before each PR's creation timestamp.

- File & directory overlap
- Commit expertise
- Author affinity
- Review network
- Reviewer latency & acceptance-rate statistics

Reviewer Recommendation Model

GradientBoostingClassifier

Estimators	400
Max depth	4
Learning rate	0.05
Subsample	0.8

- 19 numeric features + TF-IDF text similarity (over title, description, file paths)
- Trained on positive reviewer pairs with 5:1 sampled negatives
- 5-fold grouped CV (by PR) within training set only

Latency Prediction Model

RandomForestRegressor

Estimators	200
Max depth	10
Target	Log-transformed hours to first review

- Features: PR size, participant count, fastest candidate's historical latency, author's historical latency
- Calendar features (day of week, hour, weekend flag)
- Purpose: estimate reviewer responsiveness and provide the speed signal downstream

EXPERIMENTAL SETUP : INTEGRATION

Combining suitability, latency and live queue state into one queue-aware review management system.

1 PR Prioritization

Decide which PR to review first.

Rank pull requests based on urgency and suitability signals.

- Both urgency and suitability use percentile ranks.
- Higher weight is given to likely bug fixes and core authors.

2 Reviewer Assignment

Select the best reviewers for each PR.

Recommend suitable reviewers based on multiple signals.

- Combines file and directory expertise, past review history, social and network information.
- Ranks reviewers for each pull request.

3 Queue-Aware Responsiveness

Adapt to live reviewer workload.

Adjust recommendations using historical latency and current availability.

- Considers reviewer response time, author–reviewer affinity and live workload.
- Updates availability counters in real time (one reviewer capped at 15 assignments).

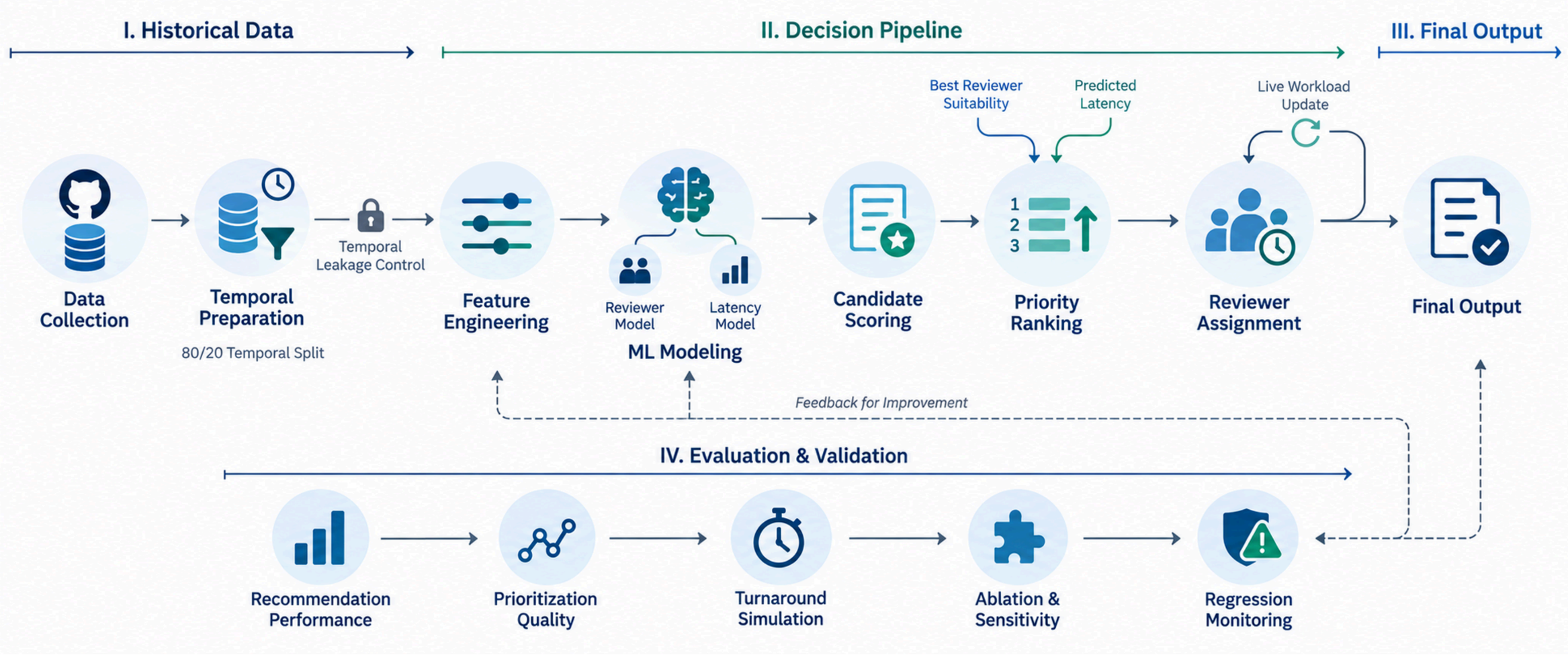
Result

A unified system that prioritizes PRs, assigns the right reviewers, and adapts to real-time workload.

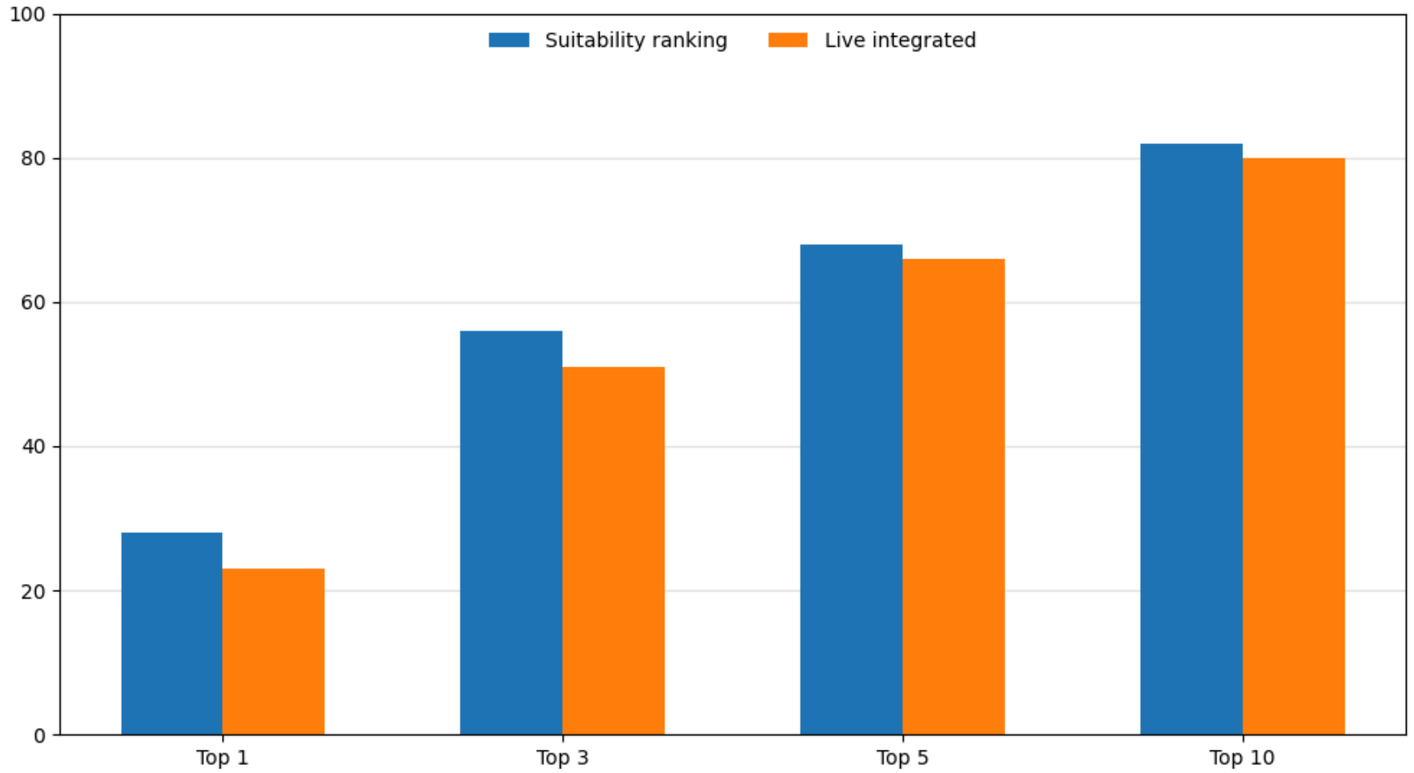
Goal

Faster and more effective code reviews.

PROPOSED METHODOLOGY



RESULTS: REVIEWER RECOMMENDATION



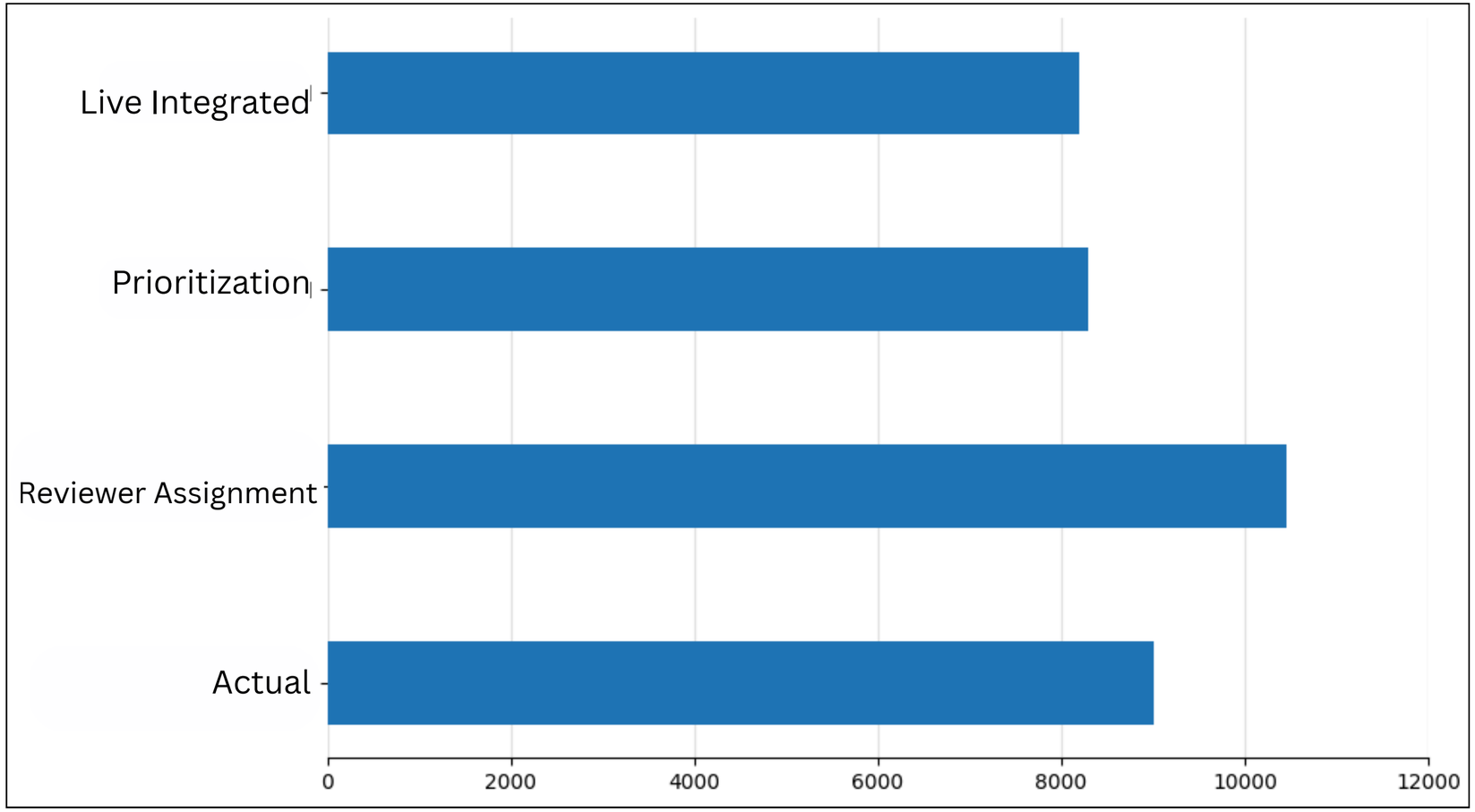
Key Result

The suitability model has higher matching accuracy. The live scoring system accepts a small accuracy cost to include workload of the reviewer.

RQ2

Higher reviewer recommendation accuracy does not necessarily translate into faster review turnaround.

RESULTS: TURNAROUND TIME



Integrated approach

8,162.9 hours

Lowest total turnaround time

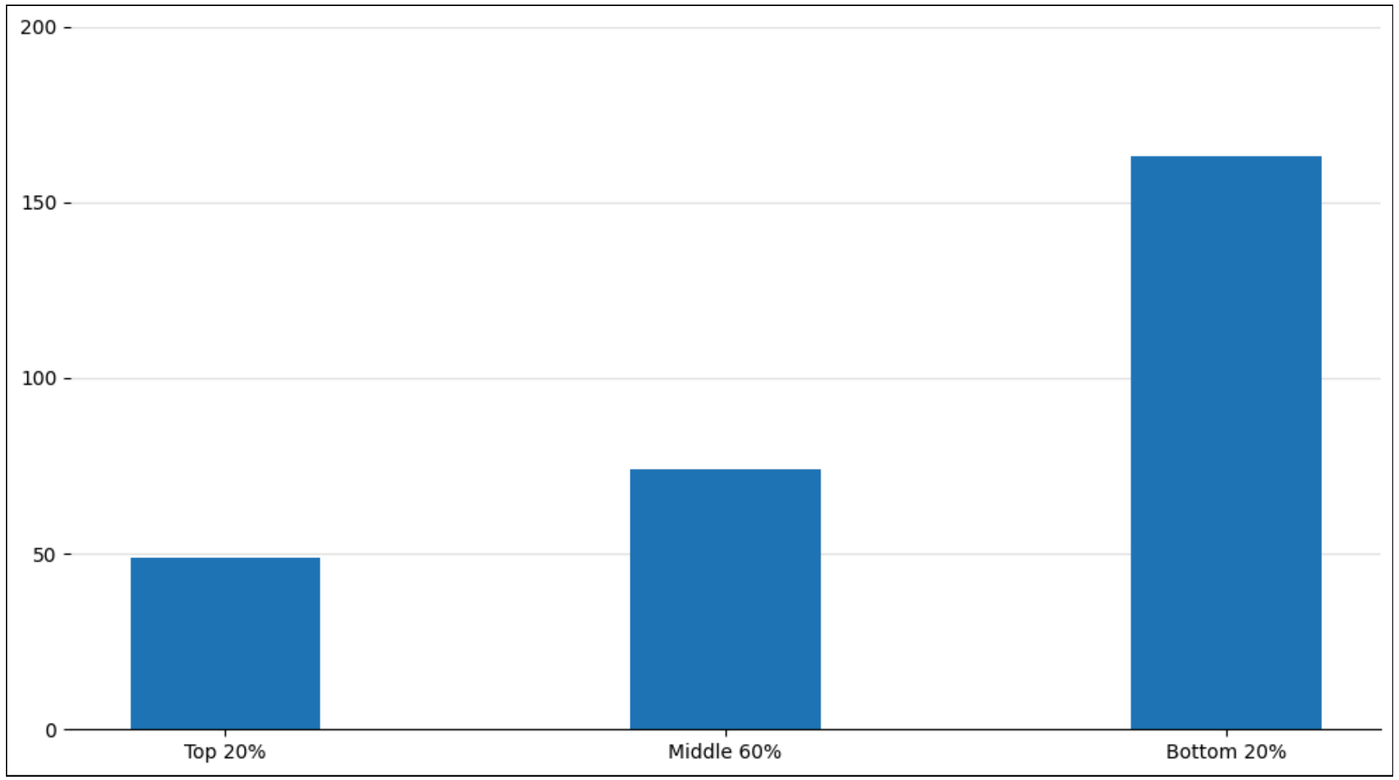
RQ1

Reviewer selection significantly affects turnaround time, but choosing suitable reviewers alone can increase delays if reviewer workload is ignored.

RQ3

Jointly optimizing reviewer assignment and PR prioritization achieves the lowest turnaround time and outperforms either approach used independently.

RESULTS: PR PRIORITIZATION



Top 20% priority
Mean latency: 48.72 hours
Median latency: 7.01 hours
High priority PRs receive attention much faster.

Middle 60% priority
Mean latency: 78.41 hours
Median latency: 21.36 hours

Bottom 20% priority
Mean latency: 183.92 hours
Median latency: 33.93 hours
Low priority PRs often wait significantly longer.

Higher priority PRs generally receive attention faster.

RESULTS : ABLATION STUDY

Each component is removed from the full reviewer recommendation system to measure its contribution.

Configuration	AUC	Top 1 (%)	Top 10 (%)	MRR	Latency (h)
Full system	0.9341	22.76	70.97	0.5326	**78.70**
Suitability only	**0.9491**	**28.21**	**72.57**	**0.5994**	91.55
Expert signals only	0.8331	15.62	56.62	0.4004	65.86
Random reviewer	0.4894	1.56	14.23	0.0949	111.47
Without responsiveness	0.9403	24.35	71.38	0.5510	86.37
Without collaboration	0.9334	22.63	70.93	0.5301	79.42
Without commit expertise	0.9365	23.99	71.67	0.5466	72.13
Without file expertise	0.9359	22.30	71.46	0.5295	94.48
Without directory expertise	0.9342	22.71	70.89	0.5336	78.67
Without specialization	0.9341	22.76	70.97	0.5327	78.68

Latency in hours. Lower values mean faster expected turnaround.

KEY FINDINGS

Full system
Lowest latency with competitive ranking quality.

Suitability only
Best on ranking metrics, but 13 h slower.

Expert signals only
Sharp drop across all ranking metrics.

Latency drivers
Responsiveness and file expertise mainly affect latency.

COMPARISON WITH PRIOR METHODS

Reproduced on the same Apache Flink evaluation setting

Reviewer recommendation

PRISM recommendation only vs. RevFinder

System	Top 1	Top 10	MRR
PRISM	**22.76%**	**81.71%**	**0.6017**
RevFinder	15.66%	63.51%	0.4214

Observed difference

+7.10 pp Top 1 | +18.20 pp Top 10 | +0.1803 MRR

PR prioritization

PRISM integrated vs. ARPrioritizer style reproduction

System	Turnaround	Spearman	NDCG@10
PRISM	**8,162.9 h**	**0.2167**	**0.2359**
ARPrioritizer	8,884 h	0.0646	0.0890

Observed difference

−721.1 h turnaround | +0.1521 Spearman | +0.1469 NDCG@10

TAKEAWAY The reproduced baselines provide direct reference points for both reviewer selection and PR ordering.

KEY FINDINGS AND CONTRIBUTIONS

Finding 1

Better reviewer matching does not necessarily mean faster reviews.

Finding 2

Reviewer workload has a major effect on queue turnaround.

Finding 3

The integrated system achieves the best simulated turnaround.

Finding 4

Integrating the two models creates coupling between reviewer recommendation and priority ordering.

LIMITATIONS & FUTURE WORK



Single repository

Evaluation is based on Apache Flink only.



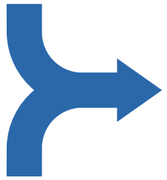
Evaluate on more repositories

Test the approach on different open source projects to verify generalizeability



First review target

The target is first review latency rather than time to final merge.



Consider Merge time

Extend the evaluation to measure the impact on actual merge time, not only the review time



Components Dependency

Improvements to reviewer recommendation can negatively affect PR prioritization quality



Joint Optimization

Improve Recommendation and Prioritization together to achieve better accuracy and ordering quality

CONCLUSION

Reviewer recommendation and PR prioritization should be considered together.

The integrated approach reduces review turnaround while maintaining competitive reviewer matching accuracy.



Lower turnaround

Integrated system achieves the best simulated review time.



Small trade-off

Slightly lower reviewer matching accuracy.

REFERENCES

[1] Gousios, G., Pinzger, M., & van Deursen, A. (2014). "An exploratory study of the pull based software development model." ICSE, 345–355.

[2] Khan, U. H. et al. (2025). "Prioritizing pull requests through dual prediction of acceptance and integrator response." Spectrum of Engineering Sciences, 3(7), 1701–1715.

[3] Mirsaeedi, E., & Rigby, P. C. (2020). "Mitigating turnover with code review recommendation: Balancing expertise, workload, and knowledge distribution." ICSE, 1183–1195.

[4] Moosareza, M. S., & Heydarnoori, A. (2024). "Automated code reviewer recommendation for pull requests." e-Informatica Software Engineering Journal, 18(1), 240108.

[5] Raskin, H., Devroey, X., & Vanderose, B. (2026). "PRECOG: Pull request prioritization and visualization." SANER 2026.

[6] Thongtanunam, P. et al. (2015). "Who should review my code? A file location based code reviewer recommendation approach for modern code review." SANER, 141–150.

[7] Yu, Y. et al. (2016). "Reviewer recommendation for pull requests in GitHub: What can we learn from code review and bug assignment?" Information and Software Technology, 74, 204–218.

[8] Yu, Y. et al. (2014). "Who should review this pull request: Reviewer recommendation to expedite crowd collaboration." APSEC, 335–342.

[9] Zanjani, M. B., Kagdi, H., & Bird, C. (2016). "Automatically recommending peer reviewers in modern code review." IEEE Transactions on Software Engineering, 42(6), 530–543.

Thank You

APPENDICES

Combining suitability, latency and live queue state into one queue-aware review management system.

1. PR Prioritization

Decide which PR to review first

$$\text{Order}(PR) = \text{urgency} \times (0.5 \times \text{speed_rank} + 0.5 \times \text{suitability_rank})$$

- Both signals use percentile ranks.
- Urgency weight starts at 1.0 (+1.0 for likely bug fixes, +0.5 for core authors).

2. Revciewer Assignment

Select the best reviewers for each PR

$$\text{Priority}(PR, r) = \alpha S + \gamma F + \delta C + \eta D + \zeta N + \iota Z + \kappa R$$

- Weights: 0.30, 0.15, 0.12, 0.08, 0.10, 0.08, 0.17
- Combines suitability, file/directory expertise, network information and responsiveness.

3. Queue-Aware Responsiveness

Adapt to live reviewer workload

$$R(r, PR) = 0.5L + 0.4A + 0.1M$$

- Uses historical latency, author affinity and live workload.
- Availability counter updates in real time (one reviewer capped at 15 assignments).

Automated Quality Gate

After each run, compare against the stored best baseline.

Spearman correlation

Top 20% latency reduction

NDCG (overall)